ammadkarar786 add class 5 and class 6 93a9bd6 · 6 minutes ago

329 lines (247 loc) · 8.46 KB

Run this setup block to create a dedicated table in your database for this lesson:

```
# Setup dedicated Employees table for window functions demonstration
conn.execute("DROP TABLE IF EXISTS employees;")
conn.execute(
    """
CREATE TABLE employees (
    employee_id INTEGER PRIMARY KEY,
    name TEXT,
    department TEXT,
    salary INT,
    hire_date DATE
);
"""
)

sample_data = [
    (1, "Alice", "Engineering", 90000, "2020-01-15"),
    (2, "Bob", "Engineering", 85000, "2021-03-22"),
    (3, "Charlie", "Engineering", 85000, "2019-07-11"),
    (4, "Diana", "Engineering", 100000, "2018-11-05"),
    (5, "Ethan", "Sales", 70000, "2020-05-19"),
    (6, "Fiona", "Sales", 75000, "2021-08-01"),
    (7, "George", "Sales", 75000, "2019-02-14"),
    (8, "Hannah", "Sales", 65000, "2022-01-10"),
    (9, "Ian", "HR", 60000, "2017-04-03"),
    (10, "Julia", "HR", 60000, "2020-09-12"),
]
```

```
conn.executemany(
    "INSERT INTO employees VALUES (?, ?, ?, ?, ?);", sample_data
)
conn.commit()

# Inspect base data
run_query("SELECT * FROM employees;")
```

Lesson Plan & Teaching Walkthrough

Topic 1: Introduction to Window (Analytic) Functions

Real-Life Analogy

Imagine a spreadsheet where you calculate the average salary of a department.

- **Standard `GROUP BY`:** Collapses 10 individual employee rows into 3 summary rows (one for each department). Individual employee names are lost.
- **Window Functions:** Keep every individual employee row visible on screen, while attaching a calculated column (like "Department Max Salary") right next to their row.

Why Do We Need Window Functions?

- Retrieve top *N* employees per department by salary without writing complex multi-level subqueries.
- Display an individual record alongside department-level aggregates (Min, Max, Avg) without collapsing data.

```
# Demonstration: Aggregating WITHOUT losing individual row details
query1 = """
SELECT
    name,
    department,
    salary,
    MAX(salary) OVER() AS overall_max_salary,
    AVG(salary) OVER() AS company_avg_salary
FROM employees;
"""
run_query(query1)
```

Topic 2: The `OVER()` Clause

Real-Life Analogy

Think of the `OVER()` clause as a **magnifying frame** you hold over your data set. By default, an empty `OVER()` looks across the entire table as one big window.

Concept Breakdown

The `OVER()` clause transforms a standard aggregate function (such as `MAX`, `MIN`, `AVG`, `SUM`) into a window function. It instructs the database engine: "*Perform this calculation across a specified window of records, but do not combine or collapse the underlying rows.*"

```
query2 = """
SELECT
    employee_id,
    name,
    salary,
    MIN(salary) OVER() AS min_company_salary,
    MAX(salary) OVER() AS max_company_salary
FROM employees;
"""
run_query(query2)
```

Topic 3: The `PARTITION BY` Clause

Real-Life Analogy

Imagine dividing a high school graduating class into separate classrooms based on their majors (Engineering, Sales, HR). `PARTITION BY` tells SQL to apply calculations **independently inside each classroom** rather than across the whole school.

Concept Breakdown

`PARTITION BY` divides the result set into distinct partitions. The window function resets its calculation at the boundary of each partition.

```
query3 = """
SELECT
    name,
    department,
    salary,
```

```
    MAX(salary) OVER(PARTITION BY department) AS dept_max_salary,
    ROUND(AVG(salary) OVER(PARTITION BY department), 2) AS dept_avg_salary
FROM employees;
"""
run_query(query3)
```

> **Teaching Tip for Students:** Ask them: "*Notice how Alice and Bob see $100,000 for `dept_max_salary` (Engineering), but Fiona sees $75,000 (Sales)? That is `PARTITION BY` at work!*"

Topic 4: The `ROW_NUMBER()` Function

Real-Life Analogy

`ROW_NUMBER()` acts like an event ticket dispenser. As people stand in line (sorted by join date), each person receives ticket #1, #2, #3, sequential and unique, with no ties allowed.

Practical Use Case

Find the first 2 employees who joined each department.

```
query4 = """
WITH RankedEmployees AS (
    SELECT
        name,
        department,
        hire_date,
        ROW_NUMBER() OVER(
            PARTITION BY department
            ORDER BY hire_date ASC
        ) AS seq_num
    FROM employees
)
SELECT *
FROM RankedEmployees
WHERE seq_num <= 2;
"""
run_query(query4)
```

Topic 5: The `RANK()` Function

Real-Life Analogy

Think of the **Olympic Leaderboard**. If two runners tie for 2nd place (silver medal), both get rank **2**. The next runner crossing the finish line gets rank **4** (bronze is skipped because two people occupied 2nd place).

Behavior

- Assigns ranks based on order.
- Duplicates get the **same rank**.
- **Skips** subsequent rank positions: `1, 2, 2, 4`.

```
query5 = """
SELECT
    name,
    department,
    salary,
    RANK() OVER(
        PARTITION BY department
        ORDER BY salary DESC
    ) AS salary_rank
FROM employees;
"""
run_query(query5)
```

Topic 6: The `DENSE_RANK()` Function

Real-Life Analogy

Imagine grading students on a strict scale: Highest grade gets **1st Rank**, tie for second gets **2nd Rank**, and the very next grade gets **3rd Rank** without skipping any numbers.

Behavior Comparison

Salary	`ROW_NUMBER()`	`RANK()`	`DENSE_RANK()`
$100,000	1	1	1
$85,000	2	2	2
$85,000	3	2	2
$80,000	4	**4** *(skips 3)*	**3** *(no skip)*

```
query6 = """
SELECT
    name,
    department,
    salary,
    ROW_NUMBER() OVER(PARTITION BY department ORDER BY salary DESC) AS row_num,
    RANK()       OVER(PARTITION BY department ORDER BY salary DESC) AS rank_val,
    DENSE_RANK() OVER(PARTITION BY department ORDER BY salary DESC) AS dense_rank_
FROM employees;
"""
run_query(query6)
```

> **Teaching Point:** Point out **Bob and Charlie** in Engineering or **Fiona and George** in Sales to demonstrate how `RANK()` jumps to 4 while `DENSE_RANK()` moves to 3.

Topic 7: The `LAG()` Function

Real-Life Analogy

Look over your shoulder at the person standing right behind you in line. `LAG()` lets you inspect values from **previous rows** relative to the current position.

Key Arguments

```
LAG(column_name, offset, default_value)
```

1. `column_name` : Column to fetch.
2. `offset` *(optional, default = 1)*: How many rows backward to look.
3. `default_value` *(optional, default = NULL)*: What to return if no previous row exists (e.g., first row).

```
query7 = """
SELECT
    name,
    hire_date,
    salary,
    LAG(salary, 1, 0) OVER(ORDER BY hire_date ASC) AS previous_employee_salary,
    salary - LAG(salary, 1, salary) OVER(ORDER BY hire_date ASC) AS salary_diff_fr
FROM employees;
```

```
"""
run_query(query7)
```

Topic 8: The `LEAD()` Function

Real-Life Analogy

Look ahead at the person standing in front of you. `LEAD()` fetches data from **following/subsequent rows**.

Practical Use Case

Compare an employee's hire date with the person hired immediately after them to calculate hiring intervals.

```
query8 = """
SELECT
    name,
    department,
    hire_date,
    LEAD(name, 1, 'None (Latest Hire)') OVER(
        PARTITION BY department
        ORDER BY hire_date ASC
    ) AS next_hired_employee,
    LEAD(hire_date, 1, 'N/A') OVER(
        PARTITION BY department
        ORDER BY hire_date ASC
    ) AS next_hire_date
FROM employees;
"""
run_query(query8)
```

Topic 9: Practical Use Case with `CASE` Statements

Real-World Business Scenario

A manager wants a salary progression analysis: Compare each employee's salary to the salary of the colleague hired immediately before them within the same department, and flag whether their salary is "**Higher**", "**Lower**", or the "**Same**".

```
query9 = """
WITH SalaryComparison AS (
    SELECT
        name,
        department,
        hire_date,
        salary,
        LAG(salary, 1, NULL) OVER(
            PARTITION BY department
            ORDER BY hire_date ASC
        ) AS prev_person_salary
    FROM employees
)
SELECT
    name,
    department,
    hire_date,
    salary,
    COALESCE(CAST(prev_person_salary AS TEXT), 'N/A (First Hire)') AS prev_salary,
    CASE
        WHEN prev_person_salary IS NULL THEN 'First Hire in Dept'
        WHEN salary > prev_person_salary THEN 'Higher'
        WHEN salary < prev_person_salary THEN 'Lower'
        ELSE 'Same'
    END AS salary_trend
FROM SalaryComparison;
"""
run_query(query9)
```